

计算机组成原理期末复习

- 1.冯诺依曼体系结构的特点以及计算机系统分层原理
- 2.计算机性能的评价指标
- 3.数的表示以及运算
- 4.指令的设计
- 5.存储器存储系统

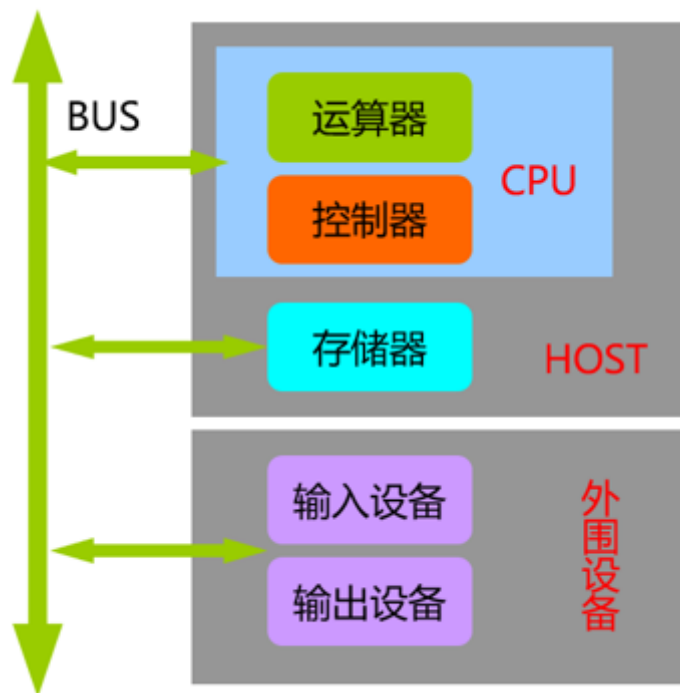
计算机组成原理期末复习

1.冯诺依曼体系结构的特点以及计算机系统分层原理

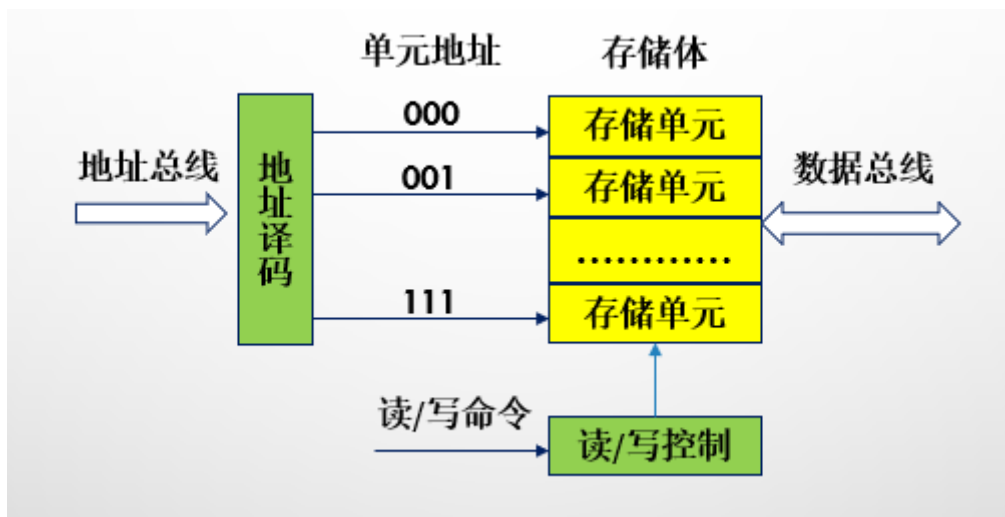
- 冯诺依曼结构计算机
 - 计算机制造的三个基本原则
 - **采用二进制逻辑**：二进制两个状态 0 和 1 更容易用逻辑电路实现
 - **程序存储执行（存储程序和程序控制）**
 - 存储程序：将**程序和运行程序**所需要的数据以**二进制**的形式存放到**存储器**中
 - 程序控制：计算机中的**控制器**逐条取出**存储器**中的指令并按顺序执行，控制各功能部件进行相应的操作，完成数据的加工处理
 - **计算机由五个部分组成**
 - 运算器
 - 控制器
 - 存储器
 - 输入设备
 - 输出设备

这套理论被称为冯诺依曼体系结构

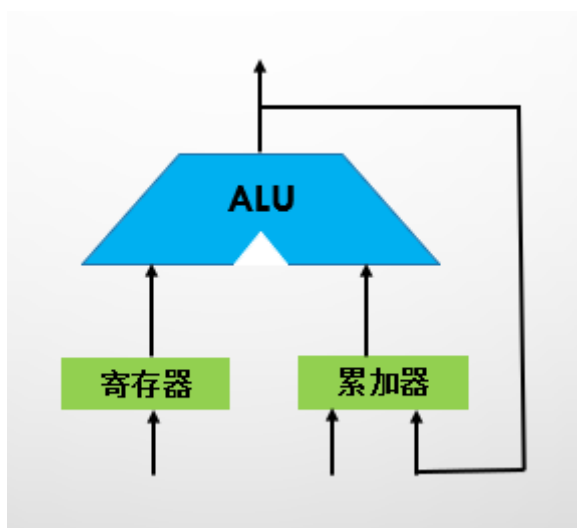
- 计算机系统的组成



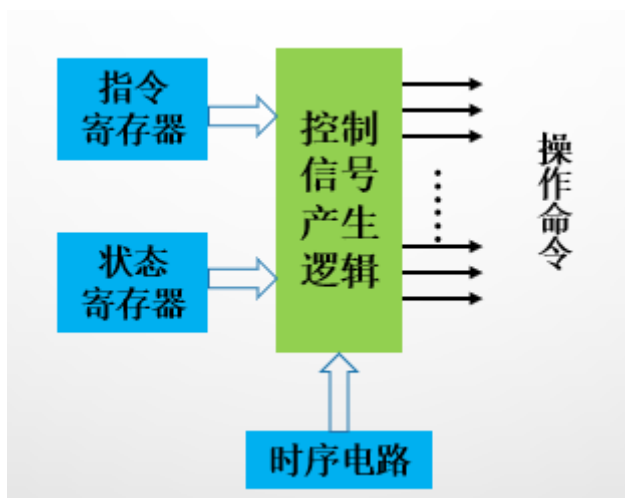
- 存储器：主要功能是**存放程序和数据**



- 运算器：用于信息加工处理的部件，它对数据进行算术运算和逻辑运算。**运算器通常由算术逻辑单元 (ALU) 和一系列寄存器组成。**

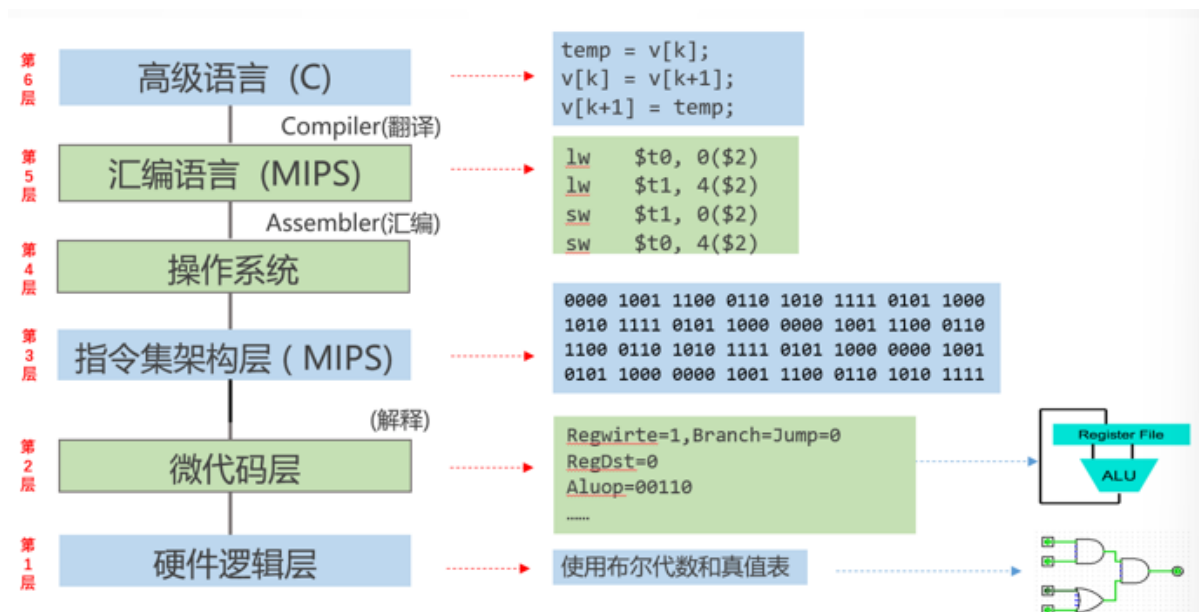


- 控制器：整个计算机的指挥中心，它可使计算机各部件协调工作。计算机中有两股信息流动：**控制流信息**和**数据流信息**。控制流信息的发源地是**控制器**，控制器产生控制流信息的依据来自3个方面：**指令寄存器**、**状态寄存器**和**时序电路**。



- 输入设备：键盘、鼠标、扫描仪、模/数（A/D）转换器等
- 输出设备：打印机、显示器、数/模（D/A）转换器等

• 计算机系统的层次结构



微代码层：该层是实际的机器层，该层的用户使用微指令编写微程序，用户编写的微程序由硬件直接执行。（只有采用微程序设计的计算机系统才有这一层）

第1、2、3层：**硬件层**，是计算机系统的基础和核心

第4、5、6层：**软件层**，第4层是**面向机器的**，第5、6层是**面向应用的**

高层是低层功能的扩展，低层是高层的基础

2.计算机性能的评价指标

- 基本性能指标
 - 字长

- 字长指CPU一次处理的数据位数，字长一般与计算机内部的寄存器、运算器、数据总线位宽相等。16位、32位、64位计算机
- 字长对计算机性能的影响
 - 字长越长计算精度越高，反之越低
 - 字长越长定点数的表示范围越大，浮点数的表示范围越大且精度也越高
- 主存容量
 - 主存能存储的最大信息量。一般用 $M \times N$ 表示， M 表示存储单元数或**字容量**； N 表示每个存储单元存储的**二进制位数（位容量）**。比如：8GB内存表示为： $8 \times 2^{30} \times 8$ ， $M = 8 \times 2^{30}$ ， $N = 8$

• **$KIB=2^{10}B$ 、 $MIB=2^{20}B$ 、 $GIB=2^{30}B$ 、 $TIB=2^{40}B$ 、 $PIB=2^{50}B$ 。**

• **$KB=10^3B$ 、 $MB=10^6B$ 、 $GB=10^9B$ 、 $TB=10^{12}B$ 、 $PB=10^{15}B$ 。**

- **$KB/KIB=1000/1024=0.97$**
- **$MB/MIB=1000000/1,048,576=0.95$**
- **$GB/GIB=1000000000/1,073,741,824=0.93$**
- **$TB/TIB=1000000000000/(1024 \times 1024 \times 1024 \times 1024)=0.91$**
- **$PB/PIB=1000000000000000/(1024 \times 1024 \times 1024 \times 1024 \times 1024)=0.88$**

- 与时间有关的性能指标

- 时钟周期（和指令周期区别）
 - 计算机中最基本、最小的时间单位
 - **主频的倒数，也称为节拍周期或T周期**
 - 例如，主频 = 1GHZ，时钟周期 = 1NS

- CPI

- **CPI(CLOCK CYCLES PER INSTRUCTION)是执行每条指令所需要的平均时钟周期数**
- **假设程序中包含的总指令条数为IC，程序执行所需的时钟周期数为M，则：**

$$CPI = \frac{m}{IC}$$

- 假设程序中有**N类指令**，第i类指令使用的频率为 P_i ，第i类指令的CPI为 CPI_i ，第i类指令的条数为 IC_i ，则：

$$CPI = \sum_{i=1}^n (CPI_i \times P_i) = \sum_{i=1}^n \left(CPI_i \times \frac{IC_i}{IC} \right)$$

- CPU时间

假设某段程序执行所需的时钟周期数为**M**，时钟周期为**T**，时钟频率为**F**，则该段程序的**CPU**时间**T_{CPU}**为：

$$T_{CPU} = m \times T = \frac{m}{f}$$
$$T_{CPU} = CPI \times IC \times T = \frac{CPI \times IC}{f}$$

CPU时间与以下3个因素紧密相关：

- ① **时钟频率F（主频）**：时钟频率越高，程序执行速度就越快。时钟频率取决于**CPU**的实现技术和工艺。
- ② **CPI**：**CPI**越小，程序执行速度就越快。**CPI**取决于计算机的实现技术和指令集结构。
- ③ **指令条数IC**：当**CPI**和时钟周期（时钟频率）固定时，程序指令条数越少，执行速度就越快。指令条数主要与指令系统的设计和编译技术有关。

○ IPC

- IPC(INSTRUCTIONS PER CYCLE)是指某个时钟周期CPU能执行的指令条数，是CPI的倒数
- IPC可以大于1，也就是说CPI可以小于1

○ MIPS

- MIPS(MILLION INSTRUCTIONS PER SECOND)为每秒执行百万条指令

• 假设某段程序中包含的总指令条数为**IC**，该段程序的**CPU**时间为**T_{CPU}**，则：

$$MIPS = \frac{IC}{T_{CPU} \times 10^6}$$
$$T_{CPU} = CPI \times IC \times T = \frac{CPI \times IC}{f}$$

$$MIPS = \frac{f}{CPI} \times 10^{-6} = IPC \times f \times 10^{-6}$$

- **计算机性能与指令的CPI和主频F有直接的关系**。主频F越高，MIPS越高；CPI越小，MIPS值越高。

○ MFLOPS

- MFLOPS(MILLION FLOATING-POINT OPERATIONS PER SECOND)每秒执行百万浮点运算次数

例题

- 例1.1：某程序的目标代码主要由4类指令组成，它们在程序中所占比例和各自CPI如表1.4所示（见教材）。请问：

① 求该程序的CPI。

② 若该CPU的主频为400MHZ，求该计算机的MIPS。

表1.4

指令类型	CPI	所占比例
算术逻辑运算指令	1	60%
访存指令	2	18%
转移指令	4	12%
其他指令	8	10%

• 答：
$$CPI = \sum_{i=1}^4 (CPI_i \times P_i) = 1 \times 0.6 + 2 \times 0.18 + 4 \times 0.12 + 8 \times 0.1 = 2.24$$

$$MIPS = \frac{f}{CPI} = \frac{400 \times 10^6}{2.24} \times 10^{-6} = 178.6$$

- 例1.2：若计算机A和B是基于相同指令集设计的两种不同类型的计算机，A的时钟周期为2NS，某程序在A上运行时的CPI为3；B的时钟周期为4NS，同一程序在B上运行时的CPI为2。请问，对这个程序而言，计算机A与B哪个更快？快多少？

• 答：

- 该程序在计算机A和计算机B上的运行时间分别为：

$$T_{CPUA} = CPI_A \times IC_A \times T_A$$

$$T_{CPUB} = CPI_B \times IC_B \times T_B$$

- 因为是同一个程序，故：

$$IC_A = IC_B = IC$$

- 因此有：

$$\frac{T_{CPUA}}{T_{CPUB}} = \frac{CPI_A \times IC_A \times T_A}{CPI_B \times IC_B \times T_B} = \frac{3 \times IC \times 2ns}{2 \times IC \times 4ns} = 0.75$$

- 即：计算机A比计算机B快，快 $1/0.75 \approx 1.3333$ 倍。

- 例1.3：设某计算机中A、B、C三类指令的CPI如表1.5所示（见教材）。现有两类不同的编译器将同一高级语言的语句编译成两种不同类型的代码序列，其中包含上述三类指令的数量如表1.6所示（见教材）。请问：

① 两种代码序列的CPI分别是多少？

② 哪种代码的执行速度快？

表1.5

指令	A	B	C
CPI	1	2	3

表1.6

代码序列	代码中3类指令的数量		
	A	B	C
1	2	1	2
2	4	1	1

答：

- 两种代码序列的CPI分别为：

$$CPI_1 = \sum_{i=1}^3 (CPI_i \times P_i) = 1 \times \left(\frac{2}{5}\right) + 2 \times \left(\frac{1}{5}\right) + 3 \times \left(\frac{2}{5}\right) = 2$$

$$CPI_2 = \sum_{i=1}^3 (CPI_i \times P_i) = 1 \times \left(\frac{4}{6}\right) + 2 \times \left(\frac{1}{6}\right) + 3 \times \left(\frac{1}{6}\right) = 1.5$$

- 两种代码序列的运行时间分别为（因为是同一个计算机，故时钟周期相同）：

$$T_{CPU1} = CPI_1 \times IC_1 \times T_1 = 2 \times 5 \times T = 10T$$

$$T_{CPU2} = CPI_2 \times IC_2 \times T_2 = 1.5 \times 6 \times T = 9T$$

- 即：第2种代码的执行速度快。尽管第2种代码的数量（6条）比第1种代码的数量（5条）多，但是执行速度却要快，原因是第2种代码中CPI小的指令所占比例更高（A类指令）。

- MAR、MDR、IR、PC等主要寄存器的宽度

- 以上几种寄存器宽度都与总线宽度有关，32位处理器中一般都是32位，64位处理器就是64位

- 机器字长、存储字长概念

- 机器字长是指计算机处理器在一次操作中可以处理的二进制数据的位数。它反映了处理器的内在数据处理能力。

- 机器字长影响处理器的计算能力、内存寻址范围、数据处理效率和指令的长度。

- 存储字长是指计算机的存储器（如RAM）中每个存储单元的位数。它表示内存系统中每个地址单元存储的数据位数。

- 存储字长影响内存访问的基本单位和存储器的组织方式。

3.数的表示以及运算

- 定点数表示

– 1、定点小数（纯小数）

- $x = x_0.x_1x_2\dots x_n$
- 例如： $x=0.1101$ （正数）， $x=1.1001$ （负数）
- x_0 表示数的符号位； $x_1 \sim x_n$ 是数值的有效部分，也称为尾数； x_1 为最高有效位。

– 2、定点整数（纯整数）

- $x = x_0x_1x_2\dots x_n$
- 例如： $x=0,1101$ （正数）， $x=1,1001$ （负数）
- x_0 表示数的符号位； $x_1 \sim x_n$ 是数值的有效部分。
- C语言中的char、short、int、long都属于定点整数。
- 定点数表示范围
- 定点数的表示范围与机器字长以及机器码（机器数的表示方法：原码、反码、补码、移码）有关。
- 若计算机字长为 $n+1$ （含1位符号位），则它可以表示 2^{n+1} 个数据状态。采用不同机器码（原码、反码、补码、移码）的定点数表示范围为：
- 教材P25-表2.4

表 2.4 定点数表示范围

	定点小数			定点整数			
	原码	反码	补码	原码	反码	补码	移码
最大正数	0.111...11 (1-2 ⁻ⁿ)	0.111...11 (1-2 ⁻ⁿ)	0.111...11 (1-2 ⁻ⁿ)	0111...11 (2 ⁿ -1)	0111...11 (2 ⁿ -1)	0111...11 (2 ⁿ -1)	1111...11 (2 ⁿ -1)
最小正数	0.000...01 (2 ⁻ⁿ)	0.000...01 (2 ⁻ⁿ)	0.000...01 (2 ⁻ⁿ)	0000...01 (1)	0000...01 (1)	0000...01 (1)	1000...01 (1)
0	0.000...00 1.000...00	0.000...00 1.111...11	0.000...00	0000...00 1000...00	0000...00 1111...11	0000...00	1000...00
最大负数	1.000...01 (-2 ⁻ⁿ)	1.111...10 (-2 ⁻ⁿ)	1.111...11 (-2 ⁻ⁿ)	1000...01 (-1)	1111...10 (-1)	1111...11 (-1)	0111...11 (-1)
最小负数	1.111...11 (-1-2 ⁻ⁿ)	1.000...00 (-1-2 ⁻ⁿ)	1.000...00 -1	1111...11 (-2 ⁿ -1)	1000...00 (-2 ⁿ -1)	1000...00 (-2 ⁿ)	0000...00 (-2 ⁿ)

- 数轴上的定点数的表示范围：

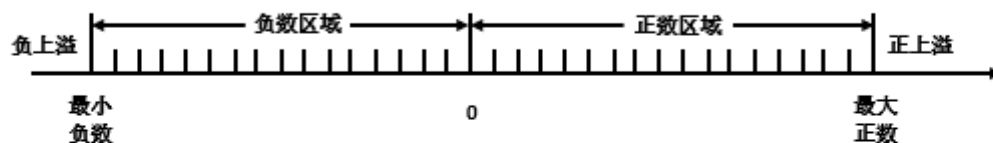


图2.5 定点数的表示范围（见教材）

- 对于定点整数，数轴上每个刻度间隔为1。
- 对于定点小数，数轴上每个刻度间隔为 2^{-n} 。
- **溢出**：当数据超出计算机所能表示的数据范围时称为溢出，数据大于最大正数时，称为**正上溢**；数据小于最小负数时，称为**负上溢**。
 - 例如：假设 $n=7$ （包括符号位共8位），整数补码的表示范围为： $-128 \sim +127$ ；如果数据=130，则发生正上溢；如果数据=-130，则发生负上溢。
 - 例如：假设 $n=7$ （包括符号位共8位），小数补码的表示范围为： $-1 \sim +(1-2^{-7})$ ；如果数据=1，则发生正上溢；如果数据=-2，则发生负上溢。
- **精度溢出**：对于小数还存在精度的问题，所有不在数轴上的小数都超出了定点小数所能表示的精度，无法表示，此时定点小数发生精度溢出，只能采用舍入的方法近似表示。
 - 例如：假设 $n=3$ （包括符号位共4位），小数补码的表示范围为： $-1 \sim +(1-2^{-3})$ ；数轴上每个刻度间隔为 2^{-3} 。数轴上的数值为： $-1, -7/8, -6/8, -5/8, -4/8, -3/8, -2/8, -1/8, 0, 1/8, 2/8, 3/8, 4/8, 5/8, 6/8, 7/8$ 。
 - 对于 $x=5/16$ （ $x=2.5/8$ ），在数轴上无法表示，发生精度溢出。因为 $x=5/16=0.0101$ ，小数点后面有4位，超出了3位（ $n=3$ ）。
 - 此时，可以采用舍入的方法近似表示；或者是 $x=0.010=2/8=4/16$ ；或者是 $x=0.011=3/8=6/16$ 。

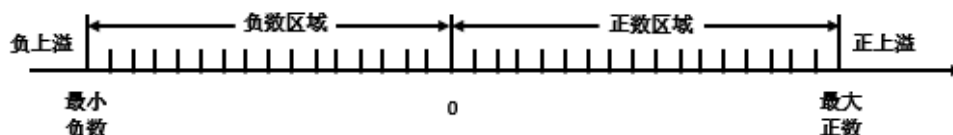


图2.5 定点数的表示范围（见教材）

- 定点数运算溢出判断

- 3种常见的溢出检测方法:

- (1) 根据操作数和运算结果的符号位是否一致进行检测

- 正数 + 正数 = 正数 不溢出
- 正数 + 正数 = 负数 溢出
- 负数 + 负数 = 负数 不溢出
- 负数 + 负数 = 正数 溢出
- 正数 + 负数 不可能产生溢出
- 负数 + 正数 不可能产生溢出

- 正正得负、负负得正时溢出

- 设 x_f 、 y_f 为操作数的符号位， s_f 为运算结果的符号位

- 溢出标志 v 检测公式: $V = x_f y_f \bar{s}_f + \bar{x}_f \bar{y}_f s_f$

- $v=0$, 没有溢出; $v=1$, 有溢出

- 如果是减法, 则溢出标志 v 的检测公式为: $V = x_f \bar{y}_f \bar{s}_f + \bar{x}_f y_f s_f$
 $- [x-y]_{补} = [x]_{补} - [y]_{补}$

- (2) 根据运算过程中最高数据位的进位与符号位的进位是否一致进行检测

- 溢出标志 v 检测公式: $V = C_f \oplus C_d$

- C_f : 符号位产生的进位信号; C_d : 最高有效位产生的进位信号

- 例3.7: $x=-111$, $y=110$, 求 $[x-y]_{补}$
- 解: $[x]_{补}=1,001$, $[y]_{补}=0,110$, $[-y]_{补}=1,010$

$$\begin{array}{r} [x]_{补} \quad 1,001 \\ + [-y]_{补} \quad 1,010 \\ \hline [x-y]_{补} \quad 10,011 \end{array}$$

$C_f = 1$ $C_d = 0$

- $C_f = 1$; $C_d = 0$

- 溢出标志 $V=1 \oplus 0=1$, 溢出 ($-7-6=-13$; 超出 $-8 \sim +7$ 的范围)

- 例3.8: $x=-011$, $y=100$, 求 $[x-y]_{补}$
- 解: $[x]_{补}=1,101$, $[y]_{补}=0,100$, $[-y]_{补}=1,110$

$$\begin{array}{r} [x]_{补} \quad 1,101 \\ + [-y]_{补} \quad 1,110 \\ \hline [x-y]_{补} \quad 11,001 \end{array}$$

$C_f = 1$ $C_d = 1$

最高有效位向前有进位
符号位向前也有进位
所以无溢出

- $C_f = 1$; $C_d = 1$

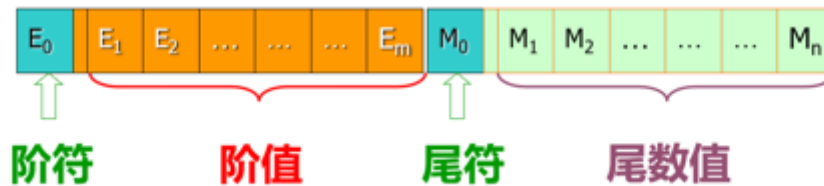
- 溢出标志 $V=1 \oplus 1=0$, 没有溢出 ($-3-4=-7$; 没有超出 $-8 \sim +7$ 的范围)

○ 定点数小数点固定，浮点数小数点位置不固定。

— $123.456 = 0.123456 \times 10^3 = 1.23456 \times 10^2 = 12.3456 \times 10^1$ （小数点位置可以浮动）

○ 浮点数表示为： $N = 2^E \times M = 2^{\pm e} \times (\pm 0.m)$

— E称为**阶码**（Exponent），为纯整数，e为阶码的数值部分；M称为**尾数**（Mantissa），为纯小数，m为尾数的数值部分。



○ 浮点数的表示范围

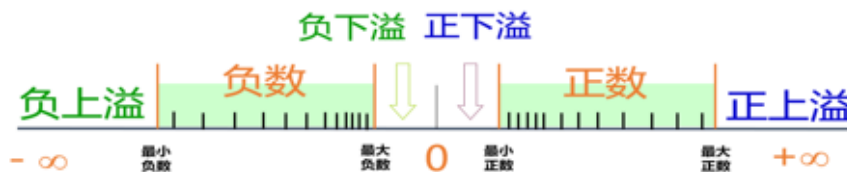


图2.7 浮点数的表示范围（见教材）

- **最大正数**：阶码为最大数、尾数为最大正数
- **最小正数**：阶码为最小数、尾数为最小正数
- **最大负数**：阶码为最小数、尾数为最大负数
- **最小负数**：阶码为最大数、尾数为最小负数
- **正上溢**（ $+\infty$ ）：浮点数大于最大正数
- **负上溢**（ $-\infty$ ）：浮点数小于最小负数
- **正下溢**（作为机器0处理）：浮点数正数小于最小正数
- **负下溢**（作为机器0处理）：浮点数负数大于最大负数
- **精度溢出**：某个浮点数在表示区间，但不在数轴刻度上，此时只能用近似数表示。
- 浮点数的数轴刻度**是不均匀的**（定点数的数轴刻度是均匀的），越往数轴的左右两端，刻度越稀疏。
- 例如**16位浮点数**，其中**阶码为5位**（含1位阶码，用移码表示）、**尾数为11位**（含1位数符，用补码表示），则：
 - 阶码的表示范围为： $-16 \sim 15$ ；**0,0000~1,1111**
 - 尾数的表示范围为： $-1 \sim +(1-2^{-10})$ ；**1.00 0000 0000~0.11 1111 1111**
 - 阶码最大数： $+15$ ；阶码最小数： -16
 - 尾数最大正数： $+(1-2^{-10})$ ；尾数最小正数： $+2^{-10}$ ；尾数最大负数： -2^{-10} ；尾数最小负数： -1
 - 浮点数最大正数： $2^{15} \times (1-2^{-10})$ ；**1,1111 0.11 1111 1111**
 - 浮点数最小正数： $2^{-16} \times 2^{-10}$ ；**0,0000 0.00 0000 0001**
 - 浮点数最大负数： $2^{-16} \times (-2^{-10})$ ；**0,0000 1.11 1111 1111**
 - 浮点数最小负数： $2^{15} \times (-1)$ ；**1,1111 1.00 0000 0000**

- 浮点数的规格化

- 十进制浮点数的规格化:

- $123.456 = 0.123456 \times 10^3$

- 尾数为纯小数，且尾数的绝对值大于等于0.1、小于1。

- 非规格化十进制浮点数: $123.456 = 0.0123456 \times 10^4$

- 非规格化十进制浮点数: $123.456 = 1.23456 \times 10^2$

- 二进制浮点数的规格化

- $1101.1001 = 0.11011001 \times 2^{100}$

- 尾数为纯小数，且尾数的绝对值大于等于0.5、小于1，即尾数的最高有效位为1（尾数用原码表示）。

- 非规格化二进制浮点数: $1101.1001 = 0.011011001 \times 2^{101}$

- 非规格化二进制浮点数: $1101.1001 = 1.1011001 \times 2^{011}$

- 非规格化的浮点数可以通过左规或右规的方法变为规格化的浮点数

- 左规: 如果尾数的绝对值小于0.5，则需要对尾数进行左移，每左移一次，尾数乘2，阶码减1，直到尾数的绝对值大于等于0.5

- $1101.1001 = 0.0011011001 \times 2^{110}$ （非规格化）

- 左移1次，得到0.011011001，阶码为101；再左移1次，得到0.11011001，阶码为100；最后得到规格化的浮点数: $0.11011001 \times 2^{100}$

- 右规: 如果尾数的绝对值大于等于1，则需要对尾数进行右移，每右移一次，尾数除2，阶码加1，直到尾数的绝对值小于1

- $1101.1001 = 11.011001 \times 2^{10}$ （非规格化）

- 右移1次，得到1.1011001，阶码为11；再右移1次，得到0.11011001，阶码为100；最后得到规格化的浮点数: $0.11011001 \times 2^{100}$

- 浮点数加减法运算步骤

- 对阶

- 求阶差

- 采用小阶向大阶看齐的方法，实现对阶

- 尾数求和

- 补码加法

- 规格化

- 舍入

- 截断法: 将移出的数据一律舍去。该方法简单，很常用。但是影响精度

- 0舍1入法: 移掉的是1，则尾数末尾加1，移掉的是0，就不加

- 溢出判断
- 例题

例题1: $x=0.1101 \times 2^{01}$; $y=(-0.1010) \times 2^{11}$ 。求 $x+y$

解: 先写出 x 和 y 的补码表示形式

$[x]_{\text{补}}=00,01$; 00.1101

$[y]_{\text{补}}=00,11$; 11.0110

1. 对阶

①求阶差:

$$\begin{array}{r} [x]_{\text{补}} - [y]_{\text{补}} = 00, 01 \\ + \quad 11, 01 \\ \hline 11, 10 \end{array}$$

11,10转化成十进制数 $\rightarrow (-2)$

所以根据小阶向大阶看齐, x 的阶码向 y 的阶码看齐, 并且 x 的阶码要加2, 尾数要右移2位。

②对阶

对阶后的 $[x]_{\text{补}}=00,11$; 00.0011

2. 尾数求和

$$\begin{array}{r} [S_x]_{\text{补}} = 00.0011 \quad \text{对阶后的}[S_x]_{\text{补}} \\ + [S_y]_{\text{补}} = 11.0110 \\ \hline 11.1001 \\ \therefore [x+y]_{\text{补}} = 00, 11; 11. 1001 \end{array}$$

3. 规格化

$[x+y]_{\text{补}}$ 的尾数是 11.1001, 符号位是 1, 尾数的最高位也是 1, 补码形式表示。

采用左规的方式, 使数据规格化: 把尾数左移一位, 同时阶码减一。

左规之后的数据: $00,10$; 11.0010

得到结果为: $x+y=(-0.1110) \times 2^{10}$

- 例3.14: 设 $x=2^{-101} \cdot (-0.101011)$, $y=2^{-010} \cdot (0.001110)$, 数的阶码为3位, 尾数为6位 (均不含符号位), 且都用补码表示, 按照补码浮点数运算步骤计算 $x+y$

解:

— 首先用补码形式表示浮点数 x 和 y (采用双符号位)

$$[x]_{\text{补}}=11\ 011, 11.010101 \quad [y]_{\text{补}}=11\ 110, 00.001110$$

— (1) 对阶: 小阶向大阶对齐, x (阶码=-5) 向 y (阶码=-2) 对齐, x 的尾数右移3位

• 对阶后的 $[x]_{\text{补}}=11\ 110, 11.111010(101)$

— (2) 尾数运算 (求和):

$$\begin{array}{r} [x]_{\text{补}} \quad 11\ 110, 11.111010(101) \\ + [y]_{\text{补}} \quad 11\ 110, 00.001110 \\ \hline [x+y]_{\text{补}} \quad 11\ 110, 00.001000(101) \end{array}$$

— (3) 尾数规格化处理: 运算结果的尾数=00.001000(101), 为非规格化数 (尾数的绝对值太小), 需要左规, 左移2次, 变为规格化数00.100010(1), 阶码减2, 阶码变为11 100

— (4) 舍入处理: 舍入处理后的尾数为: 00.100011 (0舍1入法, 末位恒置1法)

— (5) 溢出判断: 阶码的双符号位相同 (11), 没有溢出

— 运算结果: $[x+y]_{\text{补}}=11\ 100, 00.100011$, $x+y=2^{-100} \cdot (0.100011)$

— 验证:

$$x=(1/32) \cdot (-43/64), y=(1/4) \cdot (14/64), x+y=(-43/2048)+(112/2048)=69/2048$$

$$x+y=2^{-100} \cdot (0.100011)=(1/16) \cdot (35/64)=70/2048, \text{相差 } 1/2048, \text{ 原因是舍入处理引起的误差}$$

- IEEE754标准（单精度浮点数的格式）



图2.8 IEEE754 32位单精度浮点数（见教材）

- 阶码E采用移码表示，偏移量是127，而不是标准移码的128
 - IEEE754阶码的真值=**E-127**；E=0~255，真值为：-127~+128
 - 标准移码的真值=**E-128**；E=0~255，真值为：-128~+127
 - 尾数M为定点小数，其形式为1.M，1隐含了，只需保存M，节省了1位存储空间
 - 符号位S为0时表示正数，为1时表示负数
-
- 海明码、扩展海明码的编码、检错纠错过程
 - 码距（海明距离）
 - 两个编码对应二进制位不同的个数
 - 如 10101 和 00110，第1、4、5位不同，则码距为3
 - 有效编码集中任意两个码字的最小码距为该编码集的码距
 - 码距越大，抗干扰能力、纠错能力越强
 - 海明码
 - 本质上是多重奇偶校验码，既能检错也能纠错
 - 扩展海明码的编码、纠错检错过程

发送的信息是 (2-4位), 用 3 位校验位

发送的信息是 (5-11位), 用 4 位校验位

例: 用海明校验码和偶校验, 对 8 位 ASCII 字符 "M" 进行编码 (最高位为 0), 人为的引入一个错误, 说明如何找出这个错误。

"M" 的编码字为: ~~0100110010~~ 01001101

分析: 首先传入的是 8 位信息位, 则需引进 4 位校验位, 共 12 位
从右到左画表, 校验位在 1 2 4 8 处, 将信息位依次填入表

0	1	0	0	P_4	1	1	1	0	P_3	0	P_2	P_1
12	11	10	9	8	7	6	5	4	3	2	1	

其次应当确定校验位 $P_1 P_2 P_3 P_4$ 的取值

对 1 号校验位, 从 1 开始, 划 1 隔 1

0 1 0 0 P_4 1 1 0 P_3 1 P_2 P_1

此时 '1' 为奇数个, 由于进行偶校验, $\therefore P_1 = 1$

对 2 号校验位, 从 2 开始, 划 2 隔 2

0 1 0 0 P_4 1 1 0 P_3 1 P_2 1

此时 '1' 为偶数个, $\therefore P_2 = 0$

对 4 号校验位, 从 4 开始划 4, 隔 4

0 1 0 0 P_4 1 1 0 P_3 1 0 1

同理 $P_3 = 0$

对 8 号校验位, 从 8 开始划 8 隔 8

0 1 0 0 P_4 1 1 0 0 1 0 1

1. 确定好的 12 位数据 (含校验位) 你给别人发送

结果有错误怎么说

(这里人为地引入一个错误, 比如 6 号错误 (就是原来数取反))

假设收到的错误数据编码字为

0 1 0 0 1 1 0 0 0 1 0 1

12 11 10 9 8 7 6 5 4 3 2 1

然后开始找哪错了。

从 12 号校验位开始, 划 1 隔 1, 是偶数位, 正确

从 2 号校验位开始, 划 2 隔 2, 是奇数位, 错误

从 4 号 ————, 划 4 隔 4, 是奇, 错误

从 8 号 ————, 划 8 隔 8, 是偶, 正确

由此可知,

错 2, 4 号校验位同时错误

则真正的原数据在表格内是 $2+4=6$, 第 6 位错误

对 0 1 0 0 1 1 0 0 1 0 1 纠正之后为 0 1 0 0 1 1 1 0 0 1 0 1

(错误的)

(第 6 位取反)

因此发送方发送的正确 8 位数据为 0 1 0 0 1 1 0 1

以表 2.22 所示为例, 海明码 $H_n \cdots H_2 H_1$ 的各位均有对应的位置编号, 见表中第二行, 注意编号不能从 0 开始, 这是因为检错码为 0 时表示海明码无错。假定只有一位错发生, 如果是 H_1 出错, 则检错码 $G_r \cdots G_2 G_1 = 0001$; 由于检错码中只有 $G_1=1$, 表明 G_1 组有一位出错, 如果出错位是数据位应该引起多个校验组的检错位出错, 因此这里不可能是数据位出错, 而应该是 G_1 组的校验位出错, H_1 位置应该放置 G_1 组的校验位 P_1 , 故将 P_1 填写在 H_1 列的映射中; 同时在 H_1 列的 G_1 校验组行中标记“√”, 表示 H_1 参与了校验组 G_1 的校验。

表 2.22 海明编码分组规则

海明码	H_1	H_2	H_3	H_4	H_5	H_6	H_7	H_8	H_9	H_{10}	H_{11}	H_{12}	H_{13}	H_{14}	H_{15}	...
检错码 / 位置	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111	...
映射关系	P_1	P_2	D_1	P_3	D_2	D_3	D_4	P_4	D_5	D_6	D_7					...
G_1 校验组	√		√		√		√		√		√					...
G_2 校验组		√	√			√	√			√	√					...
G_3 校验组				√	√	√	√									...
G_4 校验组								√	√	√	√					...
G_5 校验组																...

根据校验分组规则以及偶校验编码定义, 可得各校验位和检错位的逻辑表达式 (注: 加撇的信息位为接收端数据):

$$\begin{aligned}
 P_1 &= D_1 \oplus D_2 \oplus D_4 & G_1 &= P'_1 \oplus D'_1 \oplus D'_2 \oplus D'_4 \\
 P_2 &= D_1 \oplus D_3 \oplus D_4 & G_2 &= P'_2 \oplus D'_1 \oplus D'_3 \oplus D'_4 \\
 P_3 &= D_2 \oplus D_3 \oplus D_4 & G_3 &= P'_3 \oplus D'_2 \oplus D'_3 \oplus D'_4
 \end{aligned}$$

◦ 例题

- (补充) 例1: 设原始数据=1010, 求该数据的海明码。假设在传输或存储过程中, 该海明码出现1位错误, 请验证海明码能够发现并纠正1位错。

— 解:

- 原始数据= $D_4D_3D_2D_1=1010$, $k=4$, 则 $r=3$
- 校验码 $P_3P_2P_1$ 如下:
 - $P_1 = D_1 \oplus D_2 \oplus D_4 = 0 \oplus 1 \oplus 1 = 0$
 - $P_2 = D_1 \oplus D_3 \oplus D_4 = 0 \oplus 0 \oplus 1 = 1$
 - $P_3 = D_2 \oplus D_3 \oplus D_4 = 1 \oplus 0 \oplus 1 = 0$
- 海明码 $H_7...H_2H_1 = D_4D_3D_2P_3D_1P_2P_1 = 1010010$
- 假设在传输或存储过程中, 海明码的第7位 (H_7) 出错, 传输或存储后的海明码= $H'_7...H'_2H'_1 = D'_4D'_3D'_2P'_3D'_1P'_2P'_1 = 0010010$
- 检错码为:
 - $G_1 = P'_1 \oplus D'_1 \oplus D'_2 \oplus D'_4 = 0 \oplus 0 \oplus 1 \oplus 0 = 1$
 - $G_2 = P'_2 \oplus D'_1 \oplus D'_3 \oplus D'_4 = 1 \oplus 0 \oplus 0 \oplus 0 = 1$
 - $G_3 = P'_3 \oplus D'_2 \oplus D'_3 \oplus D'_4 = 0 \oplus 1 \oplus 0 \oplus 0 = 1$
- $G_3G_2G_1=111$, 表示 H_7 出错, 只需要将该位取反, 即可得到正确的海明码, 说明海明码能够发现并纠正1位错。

- (补充) 例2: 假设接收到的海明码为1010110, 且假设接收到的海明码最多出现1位错误。请问该海明码对应的原始数据是多少?

— 解:

- 接收到的海明码 $H'_7...H'_2H'_1 = D'_4D'_3D'_2P'_3D'_1P'_2P'_1 = 1010110$
- 检错码为:
 - $G_1 = P'_1 \oplus D'_1 \oplus D'_2 \oplus D'_4 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$
 - $G_2 = P'_2 \oplus D'_1 \oplus D'_3 \oplus D'_4 = 1 \oplus 1 \oplus 0 \oplus 1 = 1$
 - $G_3 = P'_3 \oplus D'_2 \oplus D'_3 \oplus D'_4 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$
- $G_3G_2G_1=011$, 表示 H'_3 出错, 只需要将该位取反, 即可得到正确的海明码
- 正确的海明码= $H_7...H_2H_1 = D_4D_3D_2P_3D_1P_2P_1 = 1010010$
- 原始数据= $D_4D_3D_2D_1=1010$

重点是要记得校验码对应的分组

4.指令的设计

- 扩展操作码技术
 - 当采用统一操作码, 指令长度与各类指令的地址长度发生矛盾时, 通常采用“扩展操作码”技术加以解决。
 - 让操作码的长度随地址数的减少而增加

- 三地址指令、双地址指令、单地址指令和零地址指令

— 1、三地址指令：

- $A_3 \leftarrow (A_1) \text{ OP } (A_2)$ 如指令： **add rd,rs,rt** ; $R[rd]=R[rs]+R[rt]$
- 三地址指令的指令长度会很长；例如，设操作码OP=6位，存储容量=16KB，则存储器地址=14位；若三个地址均为存储器地址，则指令长度=6+14+14+14=48位
- 如果三地址指令中的操作数为寄存器，则可以缩短指令的长度；例如，设操作码OP=6位，寄存器数量=32个，则寄存器地址=5位；若三个地址均为寄存器地址，则指令长度=6+5+5+5=21位

— 2、双地址指令：

- $A_1 \leftarrow (A_1) \text{ OP } (A_2)$ 如指令： **ADD AL,BL** ; $AL \leftarrow AL+BL$
- 双地址指令有3种类型：
 - RR（寄存器-寄存器）型：MOV AL, BL
 - RS（寄存器-存储器）型：MOV [2000H], AL
 - SS（存储器-存储器）型：MOV [2000H], [3000H]
- RR型指令执行速度最快，使用最多；SS型指令执行速度最慢，使用最少

— 3、单地址指令：

- （1）单目运算指令
 - $A_1 \leftarrow \text{OP } (A_1)$
 - 如指令： **INC AL** ; $AL \leftarrow AL + 1$
- （2）隐含操作数的双目运算指令
 - $A_1 \leftarrow (AC) \text{ OP } (A_1)$
 - 如指令： **MUL BL** ; AL与BL相乘，结果送AX，隐含操作数AX

— 4、零地址指令：

- （1）指令本身不需要操作数
 - 如指令： **NOP** ; 空操作指令
- （2）隐含操作数的单目运算指令
 - 如指令： **DAA** ; BCD码调整指令，将AL中的内容进行BCD码调整，隐含AL



```
MOV AL, 43H
MOV BL, 29H
ADD AL, BL      ;AL=6BH
DAA             ;AL=72H      43+29=72
```

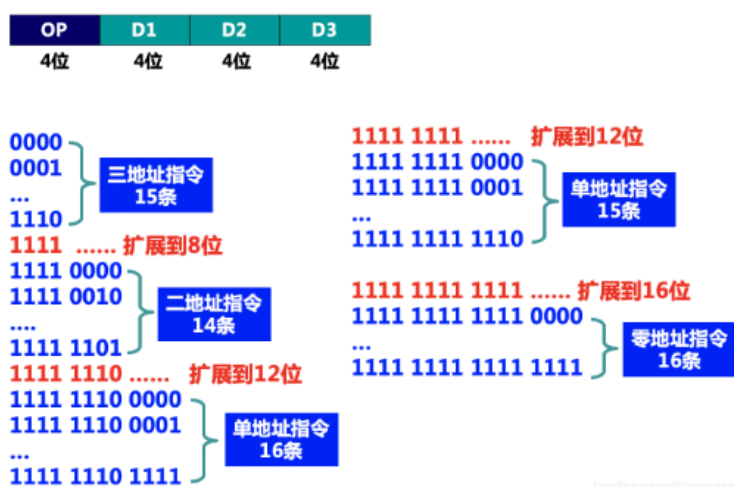
问题:(书本302页的例7.2)

假设指令字长为16位,操作数的地址码为6位,指令有零地址、一地址、二地址三种格式.采用扩展操作码技术,若二地址指令有X种,零地址指令有Y种,则一地址指令最多有几种?

书上给出的解答是:采用扩展操作码技术,操作码位数可变,则二地址、一地址和零地址的操作码长度分别为4位、10位和16位.可见二地址指令操作码每减少一种,就可多构成 2^6 种一地址指令操作码;一地址指令操作码每减少一种,就可多构成 2^6 种零地址指令操作码.好友对于划线处不太理解,我个人的解释为:减少一条二地址指令,就是将一个特定的4位操作码变为一地址指令,地址就是6位,还有10位,除去特定的4位,还有10-4位可以任意组合,所以就是 2^6 种,零地址也是一样.

所以一地址指令最多有 $(2^4 - X) * 2^6$ 种,设一地址指令有M种,则零地址指令最多有 $[(2^4 - X) * 2^6 - M] * 2^6$ 种.根据题中给出零地址有Y种,即 $Y = [(2^4 - X) * 2^6 - M] * 2^6$, 则一地址指令 $M = (2^4 - X) * 2^6 - Y * 2^6$

例: 设某指令系统, 指令字长为16位, 地址码长度为4位, 试提出一种分配方案, 使该指令系统有15条三地址指令, 14条两地址指令, 31条单地址指令, 并留有表示零地址指令的可能。



(1) [2017] 某计算机按字节编址, 指令字长固定且只有两种指令格式, 其中三地址指令 29 条, 二地址指令 107 条, 每个地址字段为 6 位, 则指令字长至少应该是 (A)。

A. 24 位 B. 26 位 C. 28 位 D. 32 位

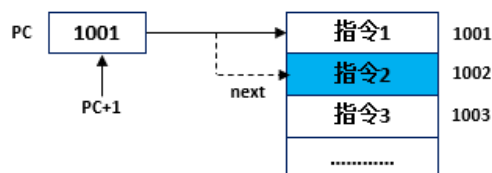
分析: 三地址指令 29 条, 说明操作码至少有 5 位, 所以剩下的操作码有 $2^5 - 29 = 3$ 种操作码给二地址指令调配。此时, 二地址能实现的指令数为 $3 * 2^6 = 192$, 所以 5 位操作码够用, 指令字长为 $5 + 6 * 3 = 23$, 取 8 的倍数为 24, 选 A

- 指令寻址方式与寻址范围的确定

- 顺序寻址

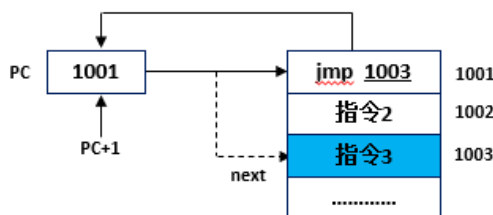
- PC 程序计数器按顺序访问主存取出指令
 - 如果内存存储单元编址单位是字节, 每一条指令的长度是 32 位, 即 4 个字节, 顺序寻址的时候 $PC + 4$; 如果是 64 位, 则 $PC + 8$

- 假设主存按照字节进行编址；如果是单字节指令，则PC+1；如果是双字节指令，则PC+2；以此类推
- 因此，PC应具有自动加1的功能（指令预取）



◦ 跳跃寻址

- 程序中有分支或转移，会改变程序的执行顺序
- 跳跃寻址方式时，下一条指令的地址由指令本身及指令需要测试的条件决定
- 无条件转移指令、条件转移指令都采用跳跃寻址方式
- 在图5.4b中，取出“jmp 1003”指令时，PC值先是变为1002（PC具有自动加1的功能）；之后根据“jmp 1003”指令，又使PC变为1003，执行完“jmp 1003”指令后，不是执行指令2，而是跳转到指令3执行



5.存储器存储系统

- 存储器的性能指标
 - 存储容量
 - 位表示法：1K×4位，表示有1K个单元（1024个单元），每个单元存放4位二进制数
 - 字节表示法：128B表示有128个单元，每个单元存放1个字节，1个字节等于8位

1KiB=1024B=2¹⁰B
 1MiB=1024KB=2²⁰B
 1GiB=1024MB=2³⁰B
 1TiB=1024GB=2⁴⁰B
 1PiB=1024TB=2⁵⁰B

1KB=1000B=10³B
 1MB=1000KiB=10⁶B
 1GB=1000MiB=10⁹B
 1TB=1000GiB=10¹²B
 1PB=1000TiB=10¹⁵B

5TB移动硬盘

5TB=5×1000×1000×1000×1000
 =5×10¹²B/(1024×1024×1024×1024)≈4.54TiB

5,000,945,201,152/(1024×1024×1024×1024)
 =4.54TiB

◦ 存取速度

- 存取时间：存储器的访问时间，指启动一次存储器操作（读或写）到该操作完成所经历的时间，读时间和写时间可能不同。
- 存取周期：连续启动两次访问操作之间的最短时间间隔
 - 存取周期 > 存取时间
 - 存取周期 = 存取时间 + 恢复时间（存储器状态的稳定恢复时间）
- 存储器带宽：单位时间内存储器所能传输的信息量
 - 存储器带宽的单位：位/秒，b/s；字节/秒，B/s
 - 存储器带宽与存取时间的长短和一次传输的数据位的多少有关
 - 存取时间越短、数据位越宽，带宽越高
- 存储器容量扩展，片选设计以及各片选地址范围的确定
 - 首先明确：对于256K×1位的概念，指的是有256K个存储单元，每个存储单元存放1位二进制数据
 - 位扩展（没有增加存储单元的数量，不用译码器进行片选）
 - 又称为字长扩展或数据总线扩展；例如可以用32个256K×1位的SRAM芯片，构成1个256K×32位的存储器
 - 地址线：18根（ $2^{18}=256K$ ），同时连接到每个SRAM芯片的地址线A（18根）
 - 数据线：32根，分别连接到每个SRAM芯片的数据线D（1根）
 - 控制线：MREQ#（相当于/MREQ）连接到每个SRAM芯片的CS端（片选信号）；R/W#（相当于-R/W）连接到每个SRAM芯片的WE端（读写控制信号）

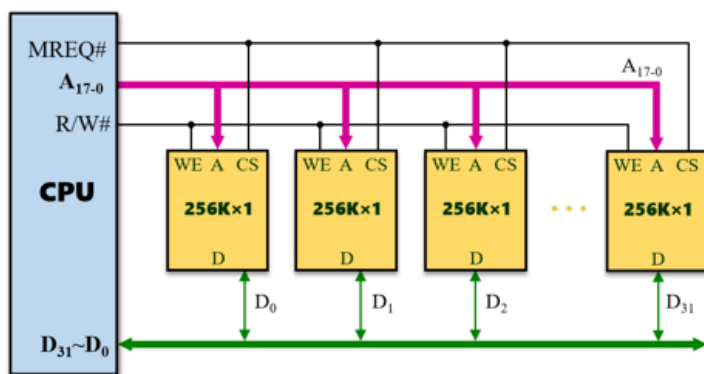


图4.24 存储器的位扩展

- 字扩展
 - 又称为容量扩展或地址总线扩展；例如可以用8个256K×8位的SRAM芯片，构成1个2M×8位的存储器

- 地址线：21根（ $2^{21}=2M$ ），其高位部分（ $A_{20} \sim A_{18}$ ）连接3-8译码器的输入，低位部分（ $A_{17} \sim A_0$ ）同时连接到每个SRAM芯片的地址线A（18根）
- 数据线：8根，同时连接到每个SRAM芯片的数据线D（8根）
- 控制线：MREQ#连接到3-8译码器的OE端；R/W#连接到每个存储器芯片的WE端
- 3-8译码器的输入： $A_{20} \sim A_{18}$ 和MREQ#
- 3-8译码器的输出： $Y_0 \sim Y_7$ ，分别连接每个SRAM的CS端

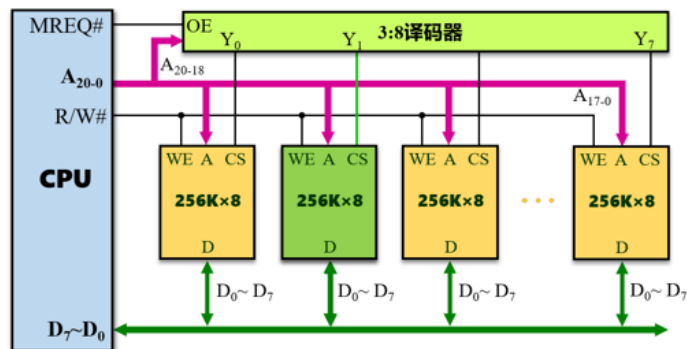
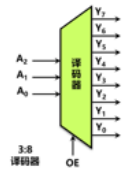


图4.25 存储器的字扩展



3-8译码器的符号

输入	输出
$A_2 A_1 A_0$ OE	$Y_7 Y_6 Y_5 Y_4 Y_3 Y_2 Y_1 Y_0$
0 0 0 1	0 0 0 0 0 0 0 1
0 0 1 1	0 0 0 0 0 0 1 0
0 1 0 1	0 0 0 0 0 1 0 0
0 1 1 1	0 0 0 0 1 0 0 0
1 0 0 1	0 0 0 1 0 0 0 0
1 0 1 1	0 0 1 0 0 0 0 0
1 1 0 1	0 1 0 0 0 0 0 0
1 1 1 1	1 0 0 0 0 0 0 0
x x x 0	x x x x x x x x

3-8译码器的输入输出关系

字位同时扩展

- 同时进行字扩展和位扩展；例如用32个256K×8位的SRAM芯片，构成1个2M×32位的存储器
- 地址线：21根（ $2^{21}=2M$ ），其高位部分（ $A_{20} \sim A_{18}$ ）连接到3-8译码器的输入，低位部分（ $A_{17} \sim A_0$ ）同时连接到每个SRAM芯片的地址线A（18根）
- 数据线：32根，分为4组（ $D_{31} \sim D_{24}$ ； $D_{23} \sim D_{16}$ ； $D_{15} \sim D_8$ ； $D_7 \sim D_0$ ），分别连接到各个SRAM芯片的数据线D（8根）
- 控制线：MREQ#连接到3-8译码器的OE端；R/W#连接到每个SRAM芯片的WE端
- 3-8译码器的输入： $A_{20} \sim A_{18}$ 和MREQ#
- 3-8译码器的输出： $Y_0 \sim Y_7$ ，分别连接每个SRAM的CS端

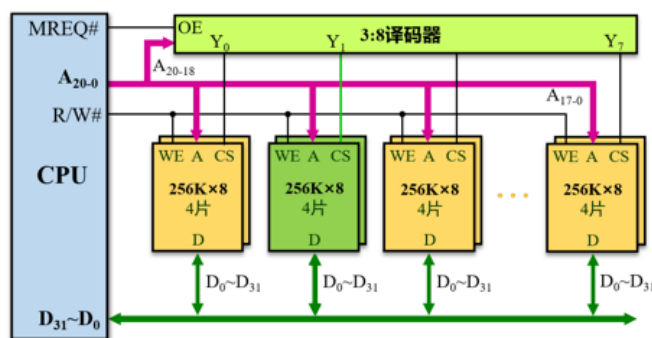
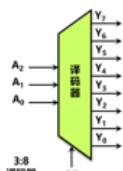


图4.26 存储器的字位扩展



3-8译码器的符号

输入	输出
$A_2 A_1 A_0$ OE	$Y_7 Y_6 Y_5 Y_4 Y_3 Y_2 Y_1 Y_0$
0 0 0 1	0 0 0 0 0 0 0 1
0 0 1 1	0 0 0 0 0 0 1 0
0 1 0 1	0 0 0 0 0 1 0 0
0 1 1 1	0 0 0 0 1 0 0 0
1 0 0 1	0 0 0 1 0 0 0 0
1 0 1 1	0 0 1 0 0 0 0 0
1 1 0 1	0 1 0 0 0 0 0 0
1 1 1 1	1 0 0 0 0 0 0 0
x x x 0	x x x x x x x x

3-8译码器的输入输出关系

cache的读写原理

- 为了提高CPU访问主存的速度，在CPU和主存之间增加一个隐藏的小容量快速SRAM，称为cache（高速缓冲存储器）
- cache读流程
 - CPU给出主存地址RA（包括块地址和块内偏移）
 - 先在cache里查找，如果找到表示命中

- 没找到就表示数据缺失，访问主存将地址为RA的主存块调入cache
 - 如果cache已经满了，还需要进行替换，并更新查找表
- cache写流程
 - CPU给出主存地址WA
 - 先在cache里查找，找到就将地址为WA的数据写到cache里，**并根据写策略决定要不要同时写入主存（写回策略不用写入主存，写直达策略需要写入主存）**
 - 找不到表示写缺失，若此时采用**写分配法**，则将地址为WA的主存块调入cache，再进行数据写入cache的操作；**如果不采用写分配法，则直接写入主存**
- cache三种地址映射
 - 全相联映射：主存的某一块可以映射到**cache的任意块**中
 - 假设：主存块地址=10位，cache块地址=5位；则主存有1024块，cache有32块（行）
 - 主存的**某一块**，可以映射到**cache的任一块**；例如主存的第0块，可以映射到cache的第0块、第1块、...、第31块

全相联映射

主存 (块)	cache (块)
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	0
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	1
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	2
.....	...
.....	...
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	30
0、1、2、3、4、5、6、7、8、9、10、.....、1022、1023	31

- 直接相联映射：主存的某一块只能映射到**cache的固定块**中

- 假设：主存块地址=10位，cache块地址=5位；则主存有1024个块，cache有32块（行）
- 主存的某一块，只能映射到cache的固定一块；例如主存的第0块，只能映射到cache的第0块；主存的第33块，只能映射到cache的第1块
-
- 地址映射公式： $i(\text{cache块}) = j(\text{主存块}) \bmod n(\text{cache块数/行数})$

直接相联映射

主存 (块)	cache (块)
0、32、64、96、128、160、192、224、.....、960、992	0
1、33、65、97、129、161、193、225、.....、961、993	1
2、34、66、98、130、162、194、226、.....、962、994	2
.....	...
.....	...
30、62、94、126、158、190、222、254、.....、990、1022	30
31、63、95、127、159、191、223、255、.....、991、1023	31

- 组相联映射：主存的某一块只能映射到cache的固定组的任意块中

- 假设：主存块地址=10位，cache块地址=5位，cache采用2路组相联映射方式；则主存有1024个块，cache分为16组、每组2块（行）、共32块（行）
- 主存的某一块，只能映射到cache的固定组（在该组中可以映射到其中的任意一块）；例如主存的第0块，只能映射到cache的第0组（可以映射到第0组的第0块或第1块）；主存的第33块，只能映射到cache的第1组（可以映射到第1组的第2块或第3块）
- 地址映射公式： $i(\text{cache组号}) = j(\text{主存块}) \bmod Q(\text{cache的组数})$

2路组相联映射

主存 (块)	cache	
	组	块
0、16、32、48、64、80、96、.....、992、1008	0	0、1
1、17、33、49、65、81、97、.....、993、1009	1	2、3
2、18、34、50、66、82、98、.....、994、1010	2	4、5
.....	...	...
.....	...	...
14、30、46、62、78、94、110、.....、1006、1022	14	28、29
15、31、47、63、79、95、111、.....、1007、1023	15	30、31

- 三种地址映射的各自特点

- 全相联映射的特点

- cache利用率高
- cache冲突率低
- 硬件成本高，只适合小容量cache使用
- cache满时，替换策略和算法较复杂

- 直接相联映射的特点

- cache利用率低，命中率低
 - cache冲突率高
 - 硬件成本低，适合大容量cache使用
 - 无需使用复杂的替换算法，直接替换冲突的数据块
 - 组相联映射的特点
 - 是直接相联映射和全相联映射的折中，既能提高命中率，又能降低查找电路的硬件成本
 - 组相联映射将cache分为若干个固定大小的组，每组有k行，称为**k-路组相联**
 -
- 虚实地址格式及其转换
- 虚拟存储器中的3种地址空间
 - 虚拟地址空间：程序员用来编写程序的地址空间
 - 主存地址空间：也称为物理地址空间、实地址空间
 - 辅存地址空间：磁盘存储器的地址空间
 - 基于页表的虚拟地址与物理地址的转换过程
 - 虚拟地址VA = 虚拟页号VPN + 虚拟页偏移VPO
 - 物理地址PA = 物理页号PPN + 物理页偏移PPO
- 将虚拟地址中的虚拟页号（VPN）与页表基址寄存器的内容（PTBR）相加，结果作为页表项的地址（PTEA），即根据相加的结果找到页表的某一行；如果该页表项（页表行）的**有效位V=1**，表明当前页的数据在主存中
- 此时直接利用页表中的物理页号（PPN）和虚拟地址中的页内偏移（VPO），就可以得到物理地址（PA）
- 如果**有效位V=0**，表示缺页，则由操作系统的缺页异常处理程序，将磁盘上对应的页载入主存中

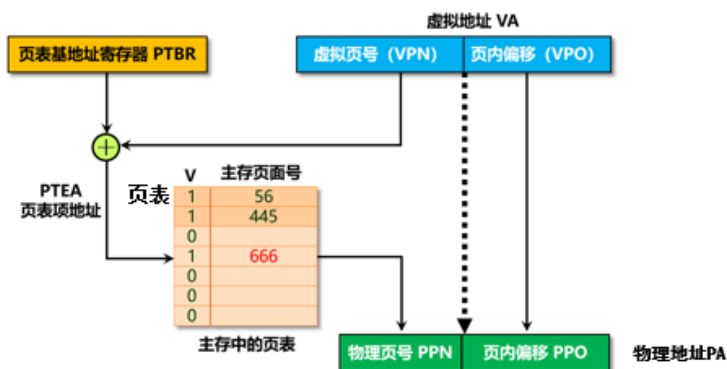


图4.48 基于页表的虚拟地址与物理地址的转换过程

例题

- 例4.6：假设主存容量=16MB，按字节寻址；虚拟存储器容量=4GB，采用页式虚拟存储器，页面大小为4KB。请回答：
- （1）计算物理页号、页内偏移、虚拟页号字段各为多少位？
- （2）计算页表中页表项的数量。
- （3）若部分页表内容如表4.10所示，求对应于虚拟地址00015240H和03FFF180H的物理地址。

虚页号	有效位	实页号	虚页号	有效位	实页号
00010H	0	002H	03FFFH	0	1
00015H	1	035H			
.....					

- 解：
- （1）
 - 主存容量=16MB，因此物理地址=24位（ $2^{24}=16M$ ）
 - 虚拟存储器容量=4GB，因此虚拟地址=32位（ $2^{32}=4G$ ）
 - 页面大小=4KB，因此页内偏移=12位（ $2^{12}=4K$ ）
 - 物理页号=物理地址-页内偏移=24-12=12位
 - 虚拟页号=虚拟地址-页内偏移=32-12=20位
- （2）
 - 页表项的数量（即页表中有多少行）与虚拟页号的位数有关；虚拟页号=20位，则页表项的数量= $2^{20}=1,048,576$ 项（行）
- （3）

20bit

12bit

 - 虚拟地址=00015240H=00015H（虚拟页号）+ 240H（页内偏移）
 - 虚拟地址=03FFF180H=03FFFH（虚拟页号）+ 180H（页内偏移）
 - 查表4.10，虚页号=00015H，对应的实页号=035H，且有效位=1，因此虚拟地址00015240H对应的物理地址=035H（物理页号）+240H（页内偏移）=035240H
 - 查表4.10，虚页号=03FFFH，对应的有效位=0，表示该虚拟页不在主存中